DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to VTU, Belagavi, Approved by AICTE & ISO 9001:2008 Certified)
Accredited by National Assessment & Accreditation Council (NAAC) with 'A' grade,
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-111

Report

On

" SMART CAMPUS INFORMATION SYSTEM APPLICATION"

Submitted by

SAKSHAM N ATHREYA(1DS25CY048)

First Semester B.E (CSE)

2025-2026

Under the guidance of

Prof.Harpreet Kaur
Assistant Professor
Dept. of CSE
DSCE, Bangalore

Department of Computer Science and Engineering
Bangalore-780111

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

DAYANANDA SAGAR COLLEGE OF ENGINEERING

Shavige Malleshwara Hills, Kumaraswamy Layout, Bangalore - 560111

Department of Computer Science & Engineering

CERTIFICATE

This is to certify that the application entitled **"SMART CAMPUS INFORMATION SYSTEM APPLICATION"** is a bonafide work carried out by **SAKSHAM N ATHREYA [1DS25CY048]** during the **Second Semester Python Programming Laboratory Sessions, Bachelor of Engineering in Computer Science and Engineering** under **Visvesvaraya Technological University, Belgaum** in the academic year **2025–26**.

Prof.Harpreet Kaur
Assistant Professor
Department of CSE,
DSCE,

Signature.....

Dr.Ramesh Babu
Vice principal & Head
Department of CSE,
DSCE

Signature.....

Name of the Examiners:	Signature with date:
1.	.....
2.	.....

Application – Smart Campus Information System

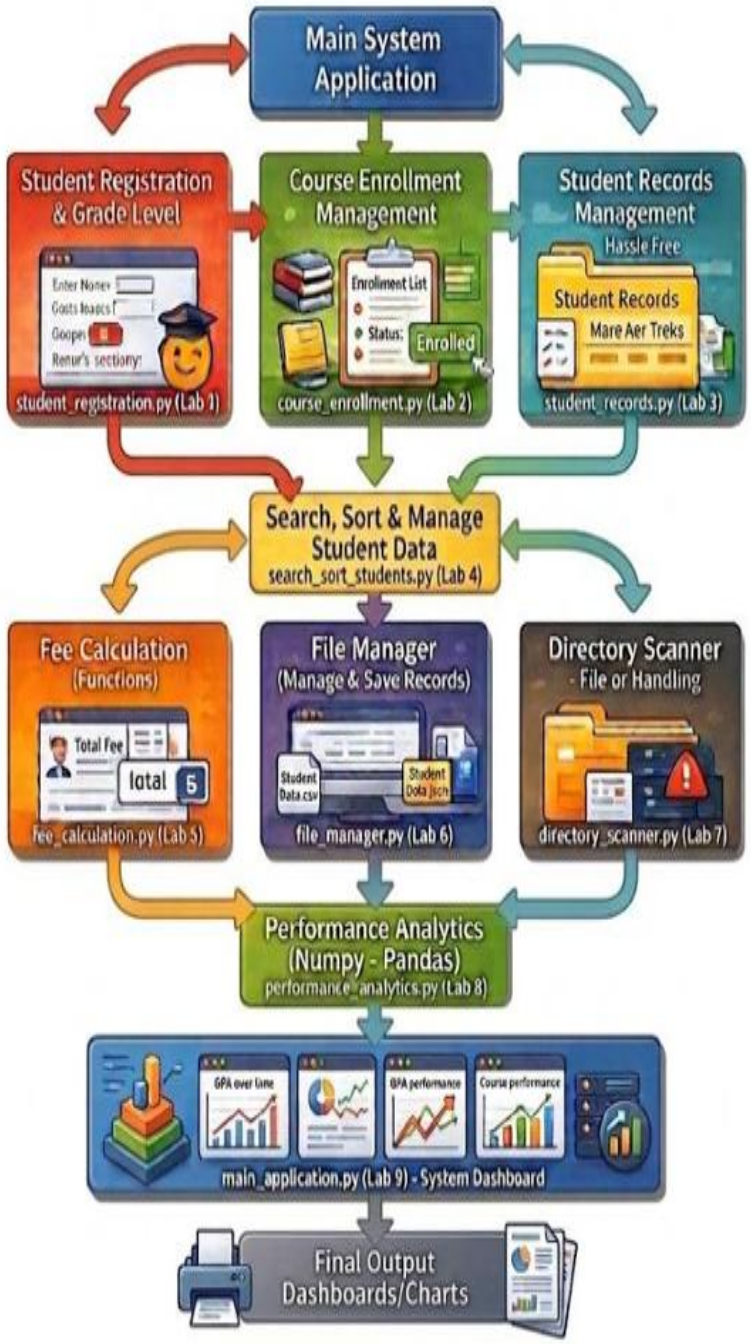
Dayananda Sagar College of Engineering plans to develop a simple Python-based Smart Campus Information System to manage student information, academic records, and performance analysis. At present, several administrative tasks such as student registration, course enrollment, record maintenance, searching student data, fee calculation, and performance analysis are often handled manually. These manual processes can be time-consuming, inefficient, and prone to errors.

To improve efficiency and accuracy, the institution intends to implement a basic software system that enables administrative staff to perform various academic management tasks digitally. The system will allow administrators to register students, manage course enrollments, maintain academic records, search and sort student data, calculate fees, store and retrieve records from files, scan directories for project files, and analyze student performance using data analytics tools.

The Smart Campus Information System will be developed gradually through Python laboratory experiments. Each experiment focuses on implementing a specific functional module, and together these modules form a complete integrated application. The final system will consist of several modules such as Student Registration, Course Enrollment Management, Student Records Management, Search and Sorting, Fee Calculation, File Management, Directory Scanning, and Performance Analytics, all integrated through a Main System Application Dashboard.

Students will develop these modules week by week, and in the final weeks they will integrate all modules to demonstrate the complete working of the Smart Campus Information System.

Overview of the Application:



Smart Campus Information System

- student_registration.py (Lab 1)
- course_enrollment.py (Lab 2)
- student_records.py (Lab 3)
- search_sort_students.py (Lab 4)
- fee_calculation.py (Lab 5)
- file_manager.py (Lab 6)
- directory_scanner.py (Lab 7)
- performance_analytics.py (Lab 8)
- main_application.py (Lab 9&10)

Smart Campus Information System (Mini Project Integration) : Demonstrate a complete Python application for the Smart Campus Information System by integrating all the modules developed from Lab 1 to Lab 8.

The final program should demonstrate the working of all components including:

- Student registration and grade evaluation
- Course enrollment management
- Student record storage and management
- Searching and sorting student data
- Fee calculation using functions
- File-based academic record management
- Directory scanning with exception handling
- Student performance analytics using NumPy, Pandas, and Matplotlib

```
# ============================================================
# SMART CAMPUS INFORMATION SYSTEM
# ============================================================

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

students = []
student_ids = []



# ============================================================
# 1. STUDENT REGISTRATION AND GRADE EVALUATION
# ============================================================

def register_student():

    while True:

        print("\n===== Student Registration =====")

        student_id = int(input("Enter Student ID: "))

        if student_id in student_ids:

            print("Student ID Already Exists")

            continue

        name = input("Enter Student Name: ")

        age = int(input("Enter Age: "))
```

```
score = float(input("Enter Exam Score: "))

# Grade Evaluation
if 90 <= score <= 100:

    grade = "A"

    remark = "Excellent"

elif score >= 75:

    grade = "B"

    remark = "Very Good"

elif score >= 60:

    grade = "C"

    remark = "Good"

elif score >= 40:

    grade = "D"

    remark = "Average"

else:

    grade = "F"

    remark = "Needs Improvement"

student = {
```

```
        "id": student_id,
        "name": name,
        "age": age,
        "score": score,
        "grade": grade,
        "remark": remark,
        "courses": [],
        "marks": {}
    }
    students.append(student)
    student_ids.append(student_id)
    print("\nStudent Registered Successfully")
    print("Grade:", grade)
    print("Remark:", remark)
    choice = input(
        "\nDo you want to add another student? (yes/no): "
    )
    if choice.lower() != "yes":
        break
```

```
# ============================================================
# 2. COURSE ENROLLMENT MANAGEMENT
# ============================================================

def enroll_courses():

    if not students:

        print("No Students Registered")

        return

    while True:

        sid = int(input("\nEnter Student ID: "))

        found = False

        for student in students:

            if student["id"] == sid:

                found = True

                print("Student Name:",
                      student["name"])

                max_courses = 5

                while True:

                    if len(student["courses"]) >= max_courses:
```

```
            print("Maximum Course Limit Reached")

            break

        course_name = input(
            "Enter Course Name (or done): "
        ).title()

        if course_name.lower() == "Done".lower():

            break

        credits = input("Enter Credits: ")

        if not credits.isdigit():

            print("Invalid Credits")

            continue

        credits = int(credits)

        if credits <= 0:

            print("Credits Must Be Positive")

            continue

        student["courses"].append(
            (course_name, credits)
        )

        print("Course Added Successfully")

    print("\n===== Enrollment Summary =====")
```

```
            total_credits = 0

            for course, credit in student["courses"]:

                print(course,
                      "-",
                      credit,
                      "Credits")

                total_credits += credit

            print("Total Courses:",
                  len(student["courses"]))

            print("Total Credits:",
                  total_credits)

        if not found:

            print("Student ID Not Found")

        choice = input(
            "\nDo you want to enroll courses for another student? (yes/no): "
        )

        if choice.lower() != "yes":

            break


# ============================================================
# 3. EVENT PARTICIPATION ANALYSIS
# ============================================================
```

```
def event_analysis():

    print("\n===== Event Participation Analysis =====")

    event_A = set(
        input(
            "Enter Event A Participants: "
        ).split()
    )

    event_B = set(
        input(
            "Enter Event B Participants: "
        ).split()
    )

    common = event_A & event_B

    all_students = event_A | event_B

    only_A = event_A - event_B

    print("\nCommon Participants:",
          common)

    print("All Participants:",
          all_students)

    print("Only Event A Participants:",
          only_A)



# ============================================================
# 4. SORT STUDENT IDS
# ============================================================
```

```
def sort_student_ids():

    if not student_ids:

        print("No Student IDs Available")

        return

    ids = student_ids.copy()

    n = len(ids)

    # Bubble Sort
    for i in range(n):

        for j in range(0, n - i - 1):

            if ids[j] > ids[j + 1]:

                ids[j], ids[j + 1] = (
                    ids[j + 1],
                    ids[j]
                )

    print("\nSorted Student IDs:")

    print(ids)


# ============================================================
# 5. SEARCH STUDENT ID
# ============================================================

def search_student_id():
```

```
if not student_ids:

    print("No Student IDs Available")

    return

target = int(
    input(
        "Enter Student ID to Search: "
    )
)

ids = sorted(student_ids)

low = 0

high = len(ids) - 1

found = False

while low <= high:

    mid = (low + high) // 2

    if ids[mid] == target:

        print(
            "Student ID Found at Position",
            mid
        )

        found = True

        break
```

```
        elif ids[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    if not found:
        print("Student ID Not Found")


# ============================================================
# 6. STUDENT FEE CALCULATION
# ============================================================

def calculate_fee(
    tuition_fee,
    hostel_fee=0,
    transport_fee=0
):
    return (
        tuition_fee +
        hostel_fee +
        transport_fee
    )


def fee_module():
    sid = int(input("Enter Student ID: "))
```

```
found = False

for student in students:

    if student["id"] == sid:

        found = True

        print("Student Name:",
              student["name"])

        tuition = float(
            input("Enter Tuition Fee: ")
        )

        hostel = float(
            input("Enter Hostel Fee: ")
        )

        transport = float(
            input("Enter Transport Fee: ")
        )

        total = calculate_fee(
            tuition,
            hostel,
            transport
        )

        print("Total Fee =", total)

if not found:

    print("Student ID Not Found")
```

```
# ============================================================
# 7. SAVE RECORDS TO TXT FILE
# ============================================================

def save_records():

    with open(
        "student_records.txt",
        "w"
    ) as file:

        for student in students:

            file.write(
                f"\nID: {student['id']}\n"
            )

            file.write(
                f"Name: {student['name']}\n"
            )

            file.write(
                f"Age: {student['age']}\n"
            )

            file.write(
                f"Score: {student['score']}\n"
            )

            file.write(
                f"Grade: {student['grade']}\n"
            )

            file.write(
```

```
            f"Remark: {student['remark']}\n"
        )

        file.write("\nCourses:\n")

        for course, credit in student["courses"]:

            file.write(
                f"{course}"
                f" - "
                f"{credit} Credits\n"
            )

        file.write(
            "--------------------------\n"
        )

    print("\nRecords Saved Successfully")


# ============================================================
# 8. READ STUDENT RECORDS
# ============================================================

def read_records():

    try:

        with open(
            "student_records.txt",
            "r"
        ) as file:

            print(
                "\n===== Student Records ====="
```

```
            )

            print(file.read())

    except FileNotFoundError:

        print("File Not Found")



# ============================================================
# 9. DIRECTORY SCANNING
# ============================================================

class MissingFileOrFolderError(Exception):

    pass


def scan_directory(path):

    try:

        if not os.path.exists(path):

            raise FileNotFoundError(
                "Invalid Directory Path"
            )

        print(f"\nScanning Directory: {path}\n")

        for root, dirs, files in os.walk(path):

            level = (
                root.replace(path, "")
                .count(os.sep)
```

```
        )

        indent = " " * 4 * level

        print(
            f"{indent}"
            f"{os.path.basename(root)}/"
        )

        sub_indent = " " * 4 * (level + 1)

        for f in files:

            print(
                f"{sub_indent}{f}"
            )

        if not files and not dirs:

            raise MissingFileOrFolderError(
                f"Empty Folder: {root}"
            )

    except FileNotFoundError as e:

        print("Error:", e)

    except MissingFileOrFolderError as e:

        print("Custom Error:", e)

    except Exception as e:

        print("Unexpected Error:", e)
```

```
# ============================================================
# 10. ENTER STUDENT MARKS
# ============================================================

def enter_marks():

    if not students:

        print("No Students Registered")

        return

    for student in students:

        print(
            f"\nEnter Marks for "
            f"{student['name']} "
            f"(ID: {student['id']})"
        )

        if not student["courses"]:

            print("No Courses Enrolled")

            continue

        student["marks"] = {}

        for course, credit in student["courses"]:

            marks = int(
                input(
                    f"{course} Marks: "
                )
```

```
            )

            student["marks"][course] = marks

    print("\nAll Student Marks Added Successfully")



# ============================================================
# 11. SAVE PERFORMANCE CSV
# ============================================================

def save_performance_csv():

    performance_data = []

    for student in students:

        row = {

            "ID": student["id"],

            "Name": student["name"]

        }

        for subject, marks in student["marks"].items():

            row[subject] = marks

        performance_data.append(row)

    df = pd.DataFrame(performance_data)

    df.to_csv(
        "student_performance.csv",
```

```
        index=False
    )

    print(
        "\nPerformance CSV File Created Successfully"
    )



# ========================================================
# 12. STUDENT PERFORMANCE ANALYSIS
# ========================================================

def performance_analysis():

    try:

        df = pd.read_csv(
            "student_performance.csv"
        )

        print("\n===== Raw Data =====")

        print(df.head())

        # Exclude ID and Name
        numeric_columns = df.columns[2:]

        print(
            "\n===== Statistical Summary ====="
        )

        print(
            df[numeric_columns].describe()
        )
```

```
scores = df[
    numeric_columns
].to_numpy()

mean_scores = np.nanmean(
    scores,
    axis=0
)

print("\n===== Mean Scores =====")

for i in range(
    len(numeric_columns)
):

    print(
        numeric_columns[i],
        ":",
        mean_scores[i]
    )

print("\n===== Top Performers =====")

for subject in numeric_columns:

    top_student = df.loc[
        df[subject].idxmax(),
        "Name"
    ]

    print(
        subject,
        ":",
        top_student
    )
```

```
# GRAPH 1
plt.figure()

plt.bar(
    numeric_columns,
    mean_scores
)

plt.title(
    "Average Scores per Subject"
)

plt.xlabel("Subjects")

plt.ylabel("Average Marks")

# GRAPH 2
plt.figure()

df.plot(
    x="Name",
    y=numeric_columns,
    kind="bar"
)

plt.title(
    "Student Performance Comparison"
)

plt.xlabel("Students")

plt.ylabel("Marks")

plt.show()
```

```
    except FileNotFoundError:
        print("CSV File Not Found")
    except Exception as e:
        print("Unexpected Error:", e)

# ============================================================
# MAIN MENU
# ============================================================

while True:
    print("\n======================================")
    print(" SMART CAMPUS INFORMATION SYSTEM ")
    print("======================================")
    print("1. Register Student")
    print("2. Enroll Courses")
    print("3. Event Participation Analysis")
    print("4. Sort Student IDs")
    print("5. Search Student ID")
    print("6. Calculate Student Fee")
    print("7. Save Records to TXT File")
```

```
print("8. Read Student Records")

print("9. Scan Directory")

print("10. Enter Student Marks")

print("11. Save Performance CSV")

print("12. Student Performance Analysis")

print("13. Exit")

choice = input("\nEnter Your Choice: ")

if choice == "1":

    register_student()

elif choice == "2":

    enroll_courses()

elif choice == "3":

    event_analysis()

elif choice == "4":

    sort_student_ids()

elif choice == "5":

    search_student_id()
```

```
elif choice == "6":

    fee_module()

elif choice == "7":

    save_records()

elif choice == "8":

    read_records()

elif choice == "9":

    path = input(
        "Enter Directory Path: "
    )

    scan_directory(path)

elif choice == "10":

    enter_marks()

elif choice == "11":

    save_performance_csv()

elif choice == "12":

    performance_analysis()

elif choice == "13":

    print("\nExiting Program...")
```

```
        break

    else:

        print("Invalid Choice")
```

OUTPUTS:

```
==================================
 SMART CAMPUS INFORMATION SYSTEM
==================================
1. Register Student
2. Enroll Courses
3. Event Participation Analysis
4. Sort Student IDs
5. Search Student ID
6. Calculate Student Fee
7. Save Records to TXT File
8. Read Student Records
9. Scan Directory
10. Enter Student Marks
11. Save Performance CSV
12. Student Performance Analysis
13. Exit

Enter Your Choice: 1

===== Student Registration =====
Enter Student ID: 101
Enter Student Name: Jay
Enter Age: 19
Enter Exam Score: 82

Student Registered Successfully
Grade: B
Remark: Very Good

Do you want to add another student? (yes/no): yes
```

```
===== Student Registration =====
Enter Student ID: 102
Enter Student Name: Andy
Enter Age: 21
Enter Exam Score: 94

Student Registered Successfully
Grade: A
Remark: Excellent

Do you want to add another student? (yes/no): yes
```

```
===== Student Registration =====
Enter Student ID: 103
Enter Student Name: Kai
Enter Age: 20
Enter Exam Score: 75

Student Registered Successfully
Grade: B
Remark: Very Good

Do you want to add another student? (yes/no): no
```

```
==================================
 SMART CAMPUS INFORMATION SYSTEM
==================================
1. Register Student
2. Enroll Courses
3. Event Participation Analysis
4. Sort Student IDs
5. Search Student ID
6. Calculate Student Fee
7. Save Records to TXT File
8. Read Student Records
9. Scan Directory
10. Enter Student Marks
11. Save Performance CSV
12. Student Performance Analysis
13. Exit

Enter Your Choice: 2

Enter Student ID: 101
Student Name: Jay
Enter Course Name (or done): Math
Enter Credits: 4
Course Added Successfully
Enter Course Name (or done): Python
Enter Credits: 3
Course Added Successfully
Enter Course Name (or done): AI
Enter Credits: 3
Course Added Successfully
Enter Course Name (or done): done

===== Enrollment Summary =====
Math - 4 Credits
Python - 3 Credits
Ai - 3 Credits
Total Courses: 3
Total Credits: 10

Do you want to enroll courses for another student? (yes/no): yes
```

```
Enter Student ID: 102
Student Name: Andy
Enter Course Name (or done): Math
Enter Credits: 4
Course Added Successfully
Enter Course Name (or done): Python
Enter Credits: 3
Course Added Successfully
Enter Course Name (or done): AI
Enter Credits: 3
Course Added Successfully
Enter Course Name (or done): done

===== Enrollment Summary =====
Math - 4 Credits
Python - 3 Credits
Ai - 3 Credits
Total Courses: 3
Total Credits: 10

Do you want to enroll courses for another student? (yes/no): yes
```

```
Enter Student ID: 103
Student Name: Kai
Enter Course Name (or done): Math
Enter Credits: 4
Course Added Successfully
Enter Course Name (or done): Python
Enter Credits: 3
Course Added Successfully
Enter Course Name (or done): AI
Enter Credits: 3
Course Added Successfully
Enter Course Name (or done): done

===== Enrollment Summary =====
Math - 4 Credits
Python - 3 Credits
Ai - 3 Credits
Total Courses: 3
Total Credits: 10

Do you want to enroll courses for another student? (yes/no): no
```

```
==================================
 SMART CAMPUS INFORMATION SYSTEM
==================================
1. Register Student
2. Enroll Courses
3. Event Participation Analysis
4. Sort Student IDs
5. Search Student ID
6. Calculate Student Fee
7. Save Records to TXT File
8. Read Student Records
9. Scan Directory
10. Enter Student Marks
11. Save Performance CSV
12. Student Performance Analysis
13. Exit

Enter Your Choice: 3

===== Event Participation Analysis =====
Enter Event A Participants: Jay Kai
Enter Event B Participants: Kai Andy

Common Participants: {'Kai'}
All Participants: {'Andy', 'Kai', 'Jay'}
Only Event A Participants: {'Jay'}
```

```
==================================
 SMART CAMPUS INFORMATION SYSTEM
==================================
1. Register Student
2. Enroll Courses
3. Event Participation Analysis
4. Sort Student IDs
5. Search Student ID
6. Calculate Student Fee
7. Save Records to TXT File
8. Read Student Records
9. Scan Directory
10. Enter Student Marks
11. Save Performance CSV
12. Student Performance Analysis
13. Exit

Enter Your Choice: 4

Sorted Student IDs:
[101, 102, 103]
```

```
==================================
 SMART CAMPUS INFORMATION SYSTEM
==================================
1. Register Student
2. Enroll Courses
3. Event Participation Analysis
4. Sort Student IDs
5. Search Student ID
6. Calculate Student Fee
7. Save Records to TXT File
8. Read Student Records
9. Scan Directory
10. Enter Student Marks
11. Save Performance CSV
12. Student Performance Analysis
13. Exit

Enter Your Choice: 5
Enter Student ID to Search: 102
Student ID Found at Position 1
```

```
==================================
 SMART CAMPUS INFORMATION SYSTEM
==================================
1. Register Student
2. Enroll Courses
3. Event Participation Analysis
4. Sort Student IDs
5. Search Student ID
6. Calculate Student Fee
7. Save Records to TXT File
8. Read Student Records
9. Scan Directory
10. Enter Student Marks
11. Save Performance CSV
12. Student Performance Analysis
13. Exit

Enter Your Choice: 6
Enter Student ID: 102
Student Name: Andy
Enter Tuition Fee: 100000
Enter Hostel Fee: 50000
Enter Transport Fee: 20000
Total Fee = 170000.0
```

```
==================================
 SMART CAMPUS INFORMATION SYSTEM
==================================
1. Register Student
2. Enroll Courses
3. Event Participation Analysis
4. Sort Student IDs
5. Search Student ID
6. Calculate Student Fee
7. Save Records to TXT File
8. Read Student Records
9. Scan Directory
10. Enter Student Marks
11. Save Performance CSV
12. Student Performance Analysis
13. Exit

Enter Your Choice: 7

Records Saved Successfully
```

```
==================================
 SMART CAMPUS INFORMATION SYSTEM
==================================
1. Register Student
2. Enroll Courses
3. Event Participation Analysis
4. Sort Student IDs
5. Search Student ID
6. Calculate Student Fee
7. Save Records to TXT File
8. Read Student Records
9. Scan Directory
10. Enter Student Marks
11. Save Performance CSV
12. Student Performance Analysis
13. Exit

Enter Your Choice: 8

===== Student Records =====

ID: 101
Name: Jay
Age: 19
Score: 82.0
Grade: B
Remark: Very Good

Courses:
Math - 4 Credits
Python - 3 Credits
Ai - 3 Credits
--------------------------
```

```
ID: 102
Name: Andy
Age: 21
Score: 94.0
Grade: A
Remark: Excellent

Courses:
Math - 4 Credits
Python - 3 Credits
Ai - 3 Credits
-------------------------

ID: 103
Name: Kai
Age: 20
Score: 75.0
Grade: B
Remark: Very Good

Courses:
Math - 4 Credits
Python - 3 Credits
Ai - 3 Credits
-------------------------
```

```
==================================
 SMART CAMPUS INFORMATION SYSTEM
==================================
1. Register Student
2. Enroll Courses
3. Event Participation Analysis
4. Sort Student IDs
5. Search Student ID
6. Calculate Student Fee
7. Save Records to TXT File
8. Read Student Records
9. Scan Directory
10. Enter Student Marks
11. Save Performance CSV
12. Student Performance Analysis
13. Exit

Enter Your Choice: 9
Enter Directory Path: C:\Users\saksh\OneDrive\Desktop\Test folder

Scanning Directory: C:\Users\saksh\OneDrive\Desktop\Test folder

Test folder/
Custom Error: Empty Folder: C:\Users\saksh\OneDrive\Desktop\Test folder
```

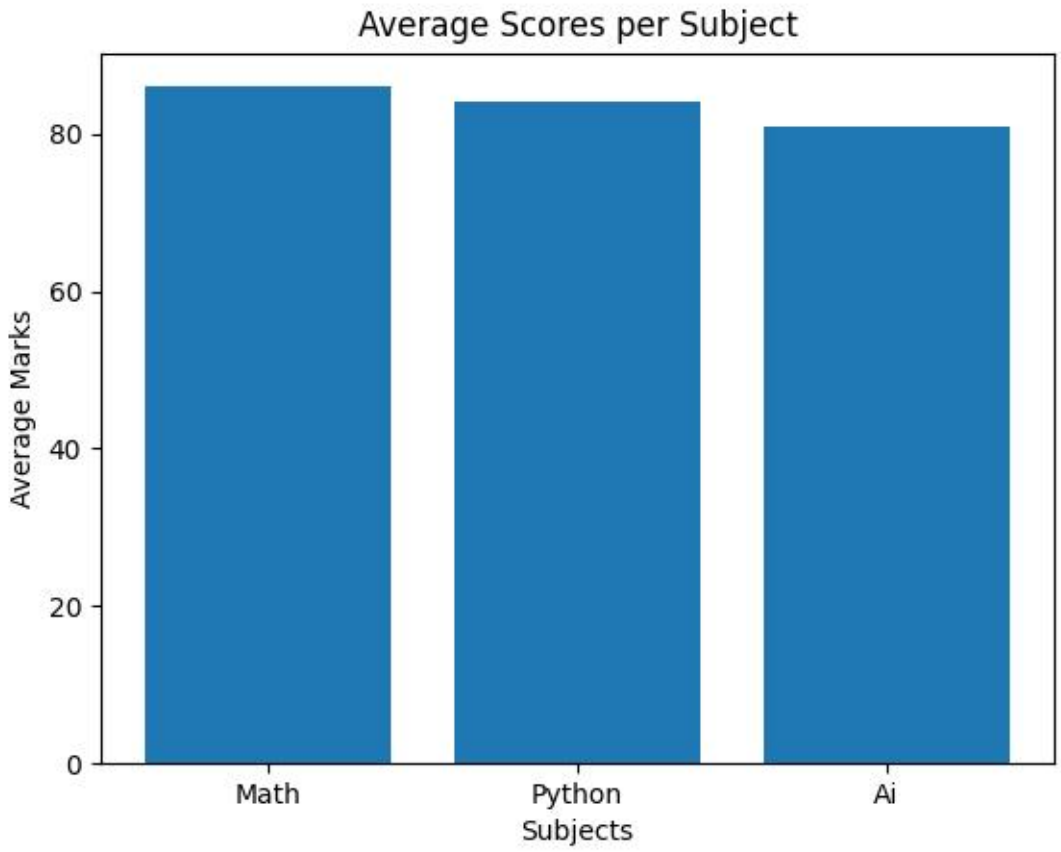
```
==================================
 SMART CAMPUS INFORMATION SYSTEM
==================================
1. Register Student
2. Enroll Courses
3. Event Participation Analysis
4. Sort Student IDs
5. Search Student ID
6. Calculate Student Fee
7. Save Records to TXT File
8. Read Student Records
9. Scan Directory
10. Enter Student Marks
11. Save Performance CSV
12. Student Performance Analysis
13. Exit

Enter Your Choice: 10

Enter Marks for Jay (ID: 101)
Math Marks: 84
Python Marks: 82
Ai Marks: 80

Enter Marks for Andy (ID: 102)
Math Marks: 99
Python Marks: 94
Ai Marks: 89

Enter Marks for Kai (ID: 103)
Math Marks: 75
Python Marks: 76
Ai Marks: 74

All Student Marks Added Successfully
```

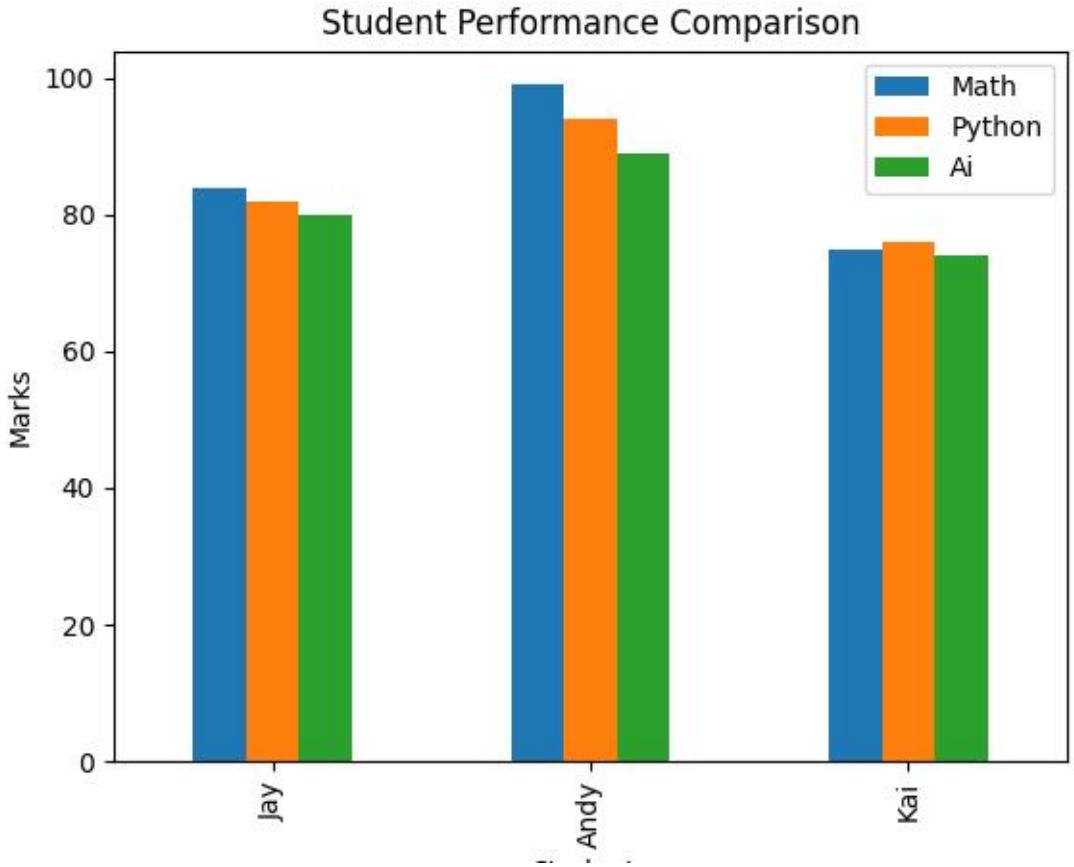
```
==================================
 SMART CAMPUS INFORMATION SYSTEM
==================================
1. Register Student
2. Enroll Courses
3. Event Participation Analysis
4. Sort Student IDs
5. Search Student ID
6. Calculate Student Fee
7. Save Records to TXT File
8. Read Student Records
9. Scan Directory
10. Enter Student Marks
11. Save Performance CSV
12. Student Performance Analysis
13. Exit

Enter Your Choice: 11

Performance CSV File Created Successfully
```

```
==================================
 SMART CAMPUS INFORMATION SYSTEM
==================================
1. Register Student
2. Enroll Courses
3. Event Participation Analysis
4. Sort Student IDs
5. Search Student ID
6. Calculate Student Fee
7. Save Records to TXT File
8. Read Student Records
9. Scan Directory
10. Enter Student Marks
11. Save Performance CSV
12. Student Performance Analysis
13. Exit

Enter Your Choice: 12

===== Raw Data =====
    ID  Name  Math  Python  Ai
0  101   Jay    84      82  80
1  102  Andy    99      94  89
2  103   Kai    75      76  74

===== Statistical Summary =====
            Math     Python         Ai
count   3.000000   3.000000   3.000000
mean   86.000000  84.000000  81.000000
std    12.124356   9.165151   7.549834
min    75.000000  76.000000  74.000000
25%    79.500000  79.000000  77.000000
50%    84.000000  82.000000  80.000000
75%    91.500000  88.000000  84.500000
max    99.000000  94.000000  89.000000

===== Mean Scores =====
Math : 86.0
Python : 84.0
Ai : 81.0
```

```
===== Top Performers =====
Math : Andy
Python : Andy
Ai : Andy
```

```
==================================
 SMART CAMPUS INFORMATION SYSTEM
==================================
1. Register Student
2. Enroll Courses
3. Event Participation Analysis
4. Sort Student IDs
5. Search Student ID
6. Calculate Student Fee
7. Save Records to TXT File
8. Read Student Records
9. Scan Directory
10. Enter Student Marks
11. Save Performance CSV
12. Student Performance Analysis
13. Exit

Enter Your Choice: 13
```


Average Scores per Subject
Average Marks
0
20
40
60
80
Math
Python
Ai
Subjects


Student Performance Comparison
Marks
0
20
40
60
80
100
Math
Python
Ai
Jay
Andy
Kai